CSE 4137

Machine Learning
Module-07
MLP: Deep Learning



Md. Shahid Uz Zaman
Adjunct Faculty
Dept of CSE, BAUET
szaman22.ruet@gmail.com

Multilayer Perceptron (MLP)

Week-13

Limitation of Single Layer Perceptron

The single layer perceptron is a very simple algorithm that can only learn linear decision boundaries. This means that it is not well suited for more complex problems where the data is not linearly separable. Additionally, the single layer perceptron is a batch learning algorithm, which means that it cannot learn from new data points incrementally. The single layer perceptron is susceptible to local minima, meaning that it may not find the global optimum solution to a problem.

Multi-layer Perceptron

A multilayer perceptron (MLP) is a type of artificial neural network that consists of multiple layers of nodes (neurons) arranged in a feedforward architecture. Here's a breakdown of its key components:

Structure

1.Input Layer: The first layer that receives input data. Each neuron in this layer represents a feature of the input data.

2.Hidden Layers: One or more layers between the input and output layers. These layers perform computations and learn to extract features from the input data. Each neuron in a hidden layer applies a weighted sum of its inputs followed by a nonlinear activation function (e.g., ReLU, sigmoid, or tanh).

3.Output Layer: The final layer that produces the output of the network. The number of neurons in this layer typically corresponds to the number of classes in a classification task or a single neuron for regression tasks.

Multilayer Perceptron (MLP)

Week-13

Characteristics

•**Feedforward Architecture**: In an MLP, information moves in one direction—from the input layer to the output layer—without any cycles or loops.

•**Activation Functions**: Each neuron uses an activation function to introduce non-linearity into the model, allowing it to learn complex patterns.

•**Training**: MLPs are trained using a method called backpropagation, which involves adjusting the weights of the connections between neurons based on the error of the network's predictions compared to the actual outputs.

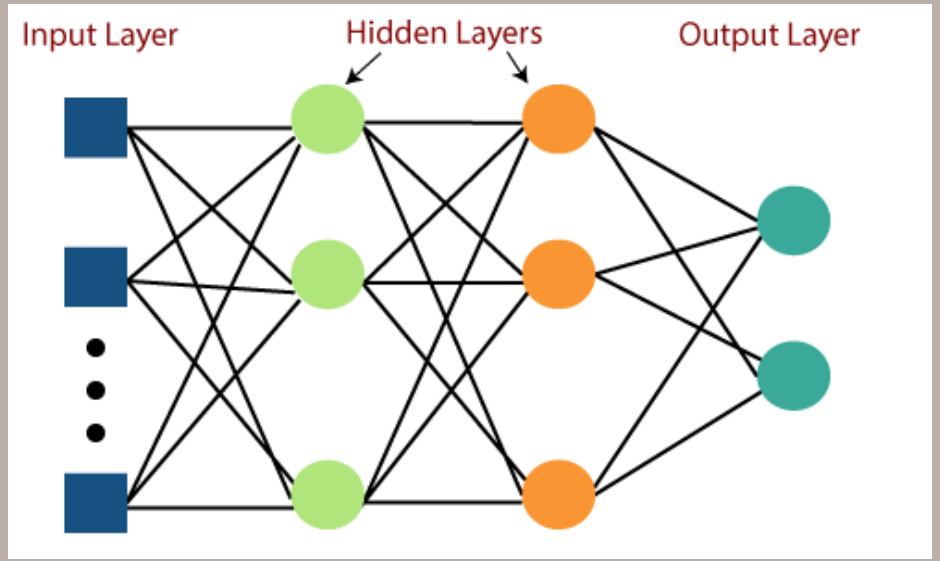
Applications

MLPs are widely used for various tasks, including:

•**Classification**: Identifying which category an input belongs to (e.g., image classification).

•**Regression**: Predicting continuous values (e.g., stock prices).

•**Function Approximation**: Learning complex functions based on input-output pairs.



Deep Learning

Week-13, c-3

Python Example:

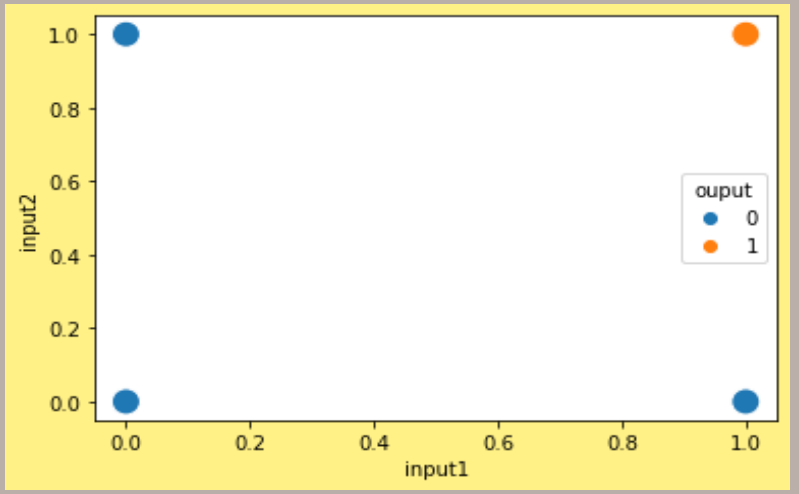
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from mlxtend.plotting import plot_decision_regions
```

```
or_data['input1']=[1,1,0,0]
or_data['input2']=[1,0,1,0]
or_data['ouput']=[1,1,1,0]
```

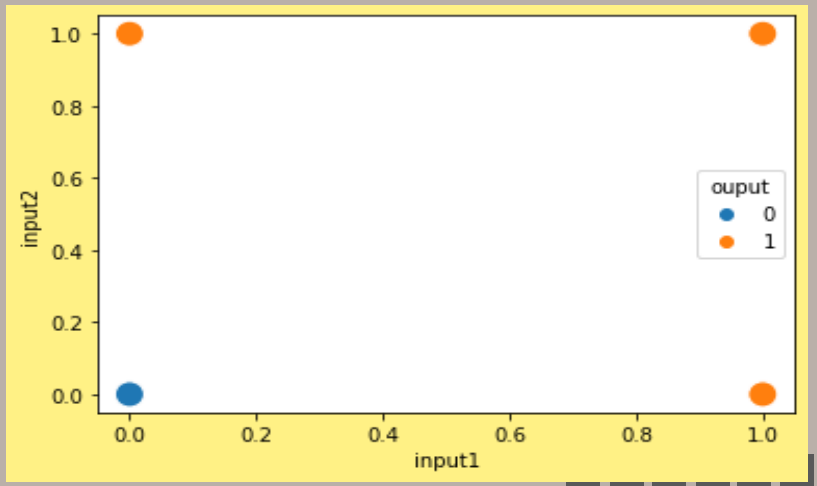
```
and_data['input1']=[1,1,0,0]
and_data['input2']=[1,0,1,0]
and_data['ouput']=[1,0,0,0]
```

```
xor_data['input1']=[1,1,0,0]
xor_data['input2']=[1,0,1,0]
xor_data['ouput']=[0,1,1,0]
```

```
sns.scatterplot(and_data['input1'],and_data['input2'],hue=and_data['ouput'],s=200)
```




```
sns.scatterplot(or_data['input1'],or_data['input2'],hue=or_data['ouput'],s=200)
```

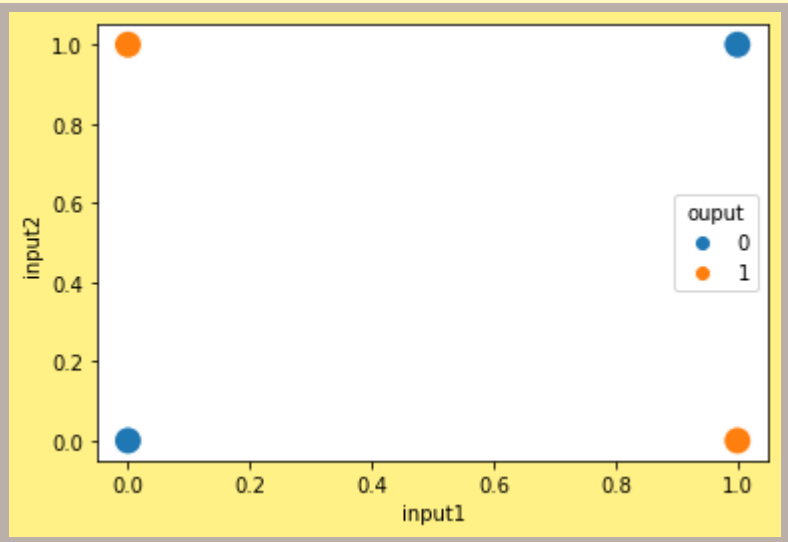


Deep Learning

Week-13, c-3

Python Example:

```
sns.scatterplot(xor_data['input1'],xor_data['input2'],hue=xor_data['ouput'],s=200)
```




```
from sklearn.linear_model import Perceptron
clf1=Perceptron()
clf2=Perceptron()
clf3=Perceptron()
```

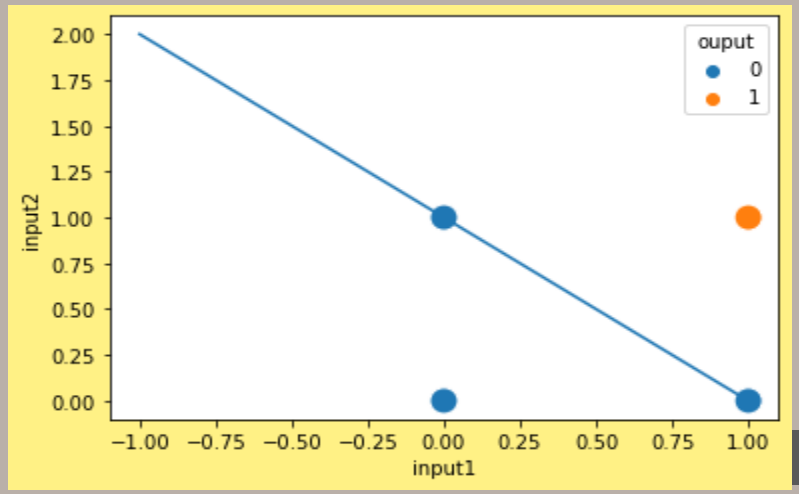
```
clf1.fit(and_data.iloc[:,0:2].values,and_data.iloc[:,-1].values)
clf2.fit(or_data.iloc[:,0:2].values,or_data.iloc[:,-1].values)
clf3.fit(xor_data.iloc[:,0:2].values,xor_data.iloc[:,-1].values)
```

```
clf1.coef_
```

```
clf1.intercept_
```

```
x=np.linspace(-1,1,5)
y=-x+1
```

```
plt.plot(x,y)
sns.scatterplot(and_data['input1'],and_data['input2'],hue=and_data['ouput'],s=200)
```



Deep Learning

Week-13, c-3

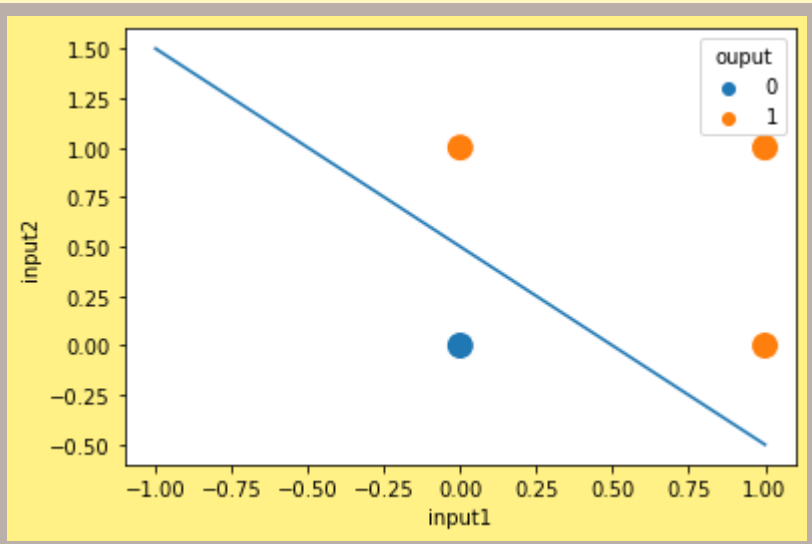
Python Example:

clf1.coef_

clf1.intercept_

x=np.linspace(-1,1,5)
y=-x+0.5

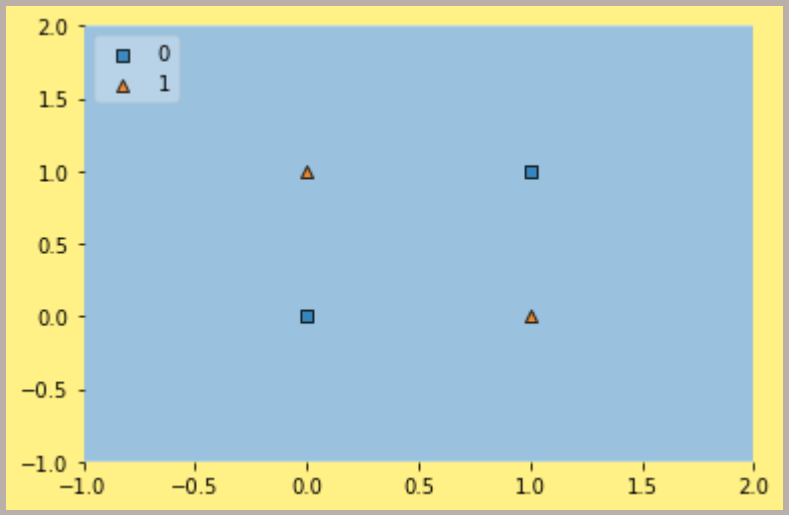
plt.plot(x,y)
sns.scatterplot(or_data['input1'],or_data['input2'],hue=or_data['ouput'],s=200)



clf1.coef_

clf1.intercept_

plot_decision_regions(xor_data.iloc[:,0:2].values,xor_data.iloc[:,-1].values, clf=clf3, legend=2)



Deep Learning

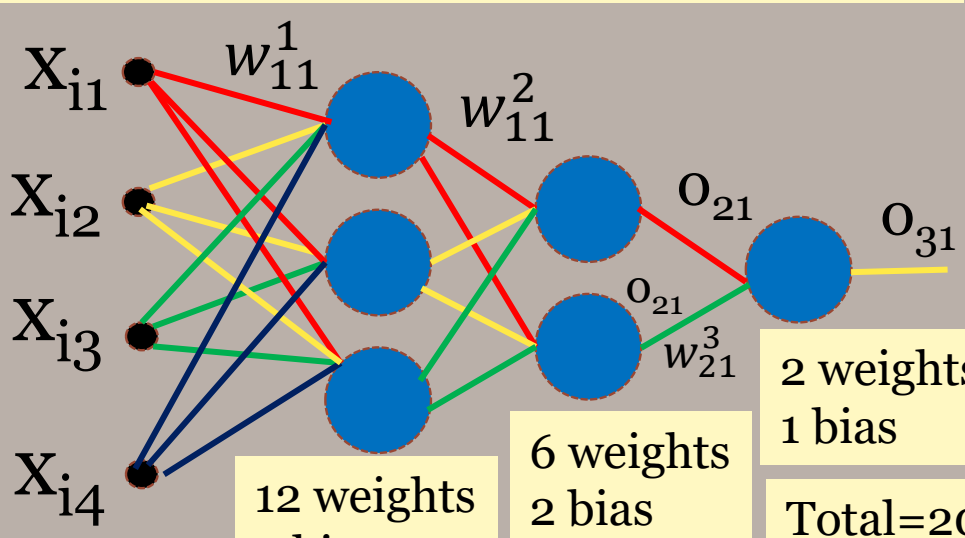
Week-13, c-3

Multilayer Perceptron (MLP)

It can solve the problem of single layer perceptron for non-linear data classification . It has several hidden layers.

MLP Notation

Here mxn (4x4) inputs where m=rows and n=features



12 weights
3 bias

6 weights
2 bias

2 weights
1 bias

Total=20 weights
Total= bias 6

Total Trainable Parameters=20 +6=24

Bias Notation: b_{ij} where i=layer and j=node
Example: b_{11} is the bias of 1st node of layer 1

Output Notation: O_{ij} where i=layer and j=node
Example: O_{11} is the output of 1st node of layer 1

Weight Notation: w_{ij}^k where k=layer entering, i=previous layer node and j=next layer node
Example: w_{42}^1 is the weight in layer 1 and it is coming from node 4 to node 2.

Deep Learning

Week-13, c-3

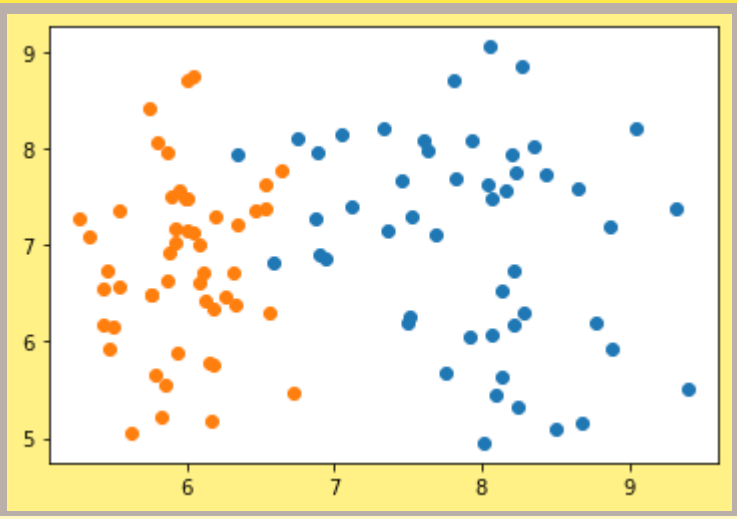
Python Example: Perceptron vs MLP

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.metrics import
confusion_matrix,accuracy_score
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier
```

```
df=pd.read_csv('F:\\python\\myplacement.csv')
```

```
df_nplaced=df[df['placed']==0]
df_placed=df[df['placed']==1]
```

```
plt.scatter(df_placed['cgpa'],df_placed['resume_score'])
plt.scatter(df_nplaced['cgpa'],df_nplaced['resume_score'])
```




```
X=df.iloc[:,0:2]
y=df.iloc[:,-1]
p=Perceptron()
p.fit(X,y)
```

```
y_pred=p.predict(X)
```

```
cm=confusion_matrix(y,y_pred)
cm
```

```
array([[39, 11],
       [14, 36]], dtype=int64)
```

```
print(accuracy_score(y,y_pred))
```

Deep Learning

Week-13, c-3

Python Example: Perceptron vs MLP

```
# use MLP)
m=MLPClassifier( hidden_layer_sizes=50,
max_iter=1500)
m.fit(X,y)
y_pred=m.predict(X)
```

```
cm=confusion_matrix(y,y_pred)
cm
```

```
array([[50,  0],
       [ 2, 48]], dtype=int64)
```

```
accuracy_score(y,y_pred)
```

Obviously MLP has better accuracy than single layer perceptron

Week-13, c-3

End of Semester

Best of Luck